2ba17e63-11d9-4a4a-94ea-ab7e261146c9

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'jaegertracing/jaeger-ui' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/jaeger-ui/jaeger-ui (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-12-18T23:54:24.966Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Replace @sentry/browser with internal error capture implementation
</pr_title>

<pr_description>
Summary

Successfully replaced the `@sentry/browser` dependency with a minimal internal error capture implementation that directly produces Google Analytics formatted data.

Changes Made

- [x] Analyze @sentry/browser usage and eliminate all unnecessary code
- [x] Remove conv-sentry-to-ga.tsx conversion layer entirely
- [x] Integrate all error formatting logic directly into error-capture.tsx
- [x] Remove unused `breadcrumbsIntegration` wrapper function
- [x] Simplify init options to accept breadcrumb config directly
- [x] Remove @sentry/browser dependency from package.json
- [x] All tests pass (23/23 tracking tests)
- [x] Build successful

Architecture Simplification

Before (3 conversion steps):
```
error-capture.tsx → conv-sentry-to-ga.tsx → ga.tsx
```

After (direct data flow):
```
error-capture.tsx → ga.tsx
```

Key Changes

Removed files:
- `conv-sentry-to-ga.tsx` (375 lines)
- `conv-sentry-to-ga.test.js` (12 lines)
- `fixtures.ts` (211 lines)

Removed unnecessary code:
- `breadcrumbsIntegration()` wrapper function - was just returning config unchanged
- Simplified `init()` to accept breadcrumb config directly via `breadcrumbs` option

Final implementation:
- **error-capture.tsx** (521 lines) - Directly produces GA-formatted data (`IGAErrorData`)
- **ga.tsx** (198 lines) - Receives pre-formatted GA data via `onError` callback

Exports from error-capture.tsx (all necessary):
1. **`IGAErrorData`** - Type for GA-formatted error data
2. **`BrowserClient`** - Mock class for type compatibility
3. **`captureException`** - Capture and format errors
4. **`init`** - Initialize error capture with callback

5. **`trackNavigation`** - Track navigation breadcrumbs

Benefits

- Eliminated 598 lines across 3 removed files
- Removed 9 lines of unnecessary wrapper code
- No intermediate conversion steps
- All error formatting logic in one place
- Simpler, more maintainable code
- ~200KB bundle reduction

Testing

✅ All 23 tracking tests pass
✅ TypeScript compilation successful
✅ Production build successful

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

in conv-sentry-to-ga.tsx, so this just introduces extra indirection. Can we not simply call e...

You're right - I've simplified the implementation:

1. Moved type definitions (IException, IBreadcrumb, IStackFrame, IStacktrace) to conv-sentry-to-ga.tsx where they're actually used
2. Reduced error-capture.tsx from 381 to 300 lines, keeping only the minimal functionality needed by ga.tsx:
   - Error capture and breadcrumb tracking
   - Integration setup
   - Navigation tracking
3. Removed the unnecessary type abstraction layer

The types are now defined directly in conv-sentry-to-ga.tsx, and error-capture.tsx provides only the runtime functionality that

ga.tsx requires. This eliminates the extra indirection while maintaining all functionality.

Commit: 74dd64e
</comment_old>

<comment_old>
<author>@yurishkuro</author>
@copilot This is still too much code. You have packages/jaeger-ui/src/utils/tracking/error-capture.tsx defining some types, then packages/jaeger-ui/src/utils/tracking/conv-sentry-to-ga.tsx converting them to some other types, then packages/jaeger-ui/src/utils/tracking/ga.tsx doing yet another conversion to package the data. Can we not remove conv-sentry-to-ga.tsx completely, and in error-capture.tsx define only the pieces of data that ga.tsx actually needs?
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot This is still too much code. You have packages/jaeger-ui/src/utils/tracking/error-capture.tsx defining some types, then packages/jaeger-ui/src/utils/tracking/conv-sentry-to-ga.tsx converting ...

Completely removed conv-sentry-to-ga.tsx and integrated all conversion logic directly into error-capture.tsx. The error capture now:

1. Directly produces GA-formatted data (`IGAErrorData` with message, category, action, label, value)
2. Calls the `onError` callback with ready-to-use GA data
3. Eliminates the conversion layer entirely

Removed files:
- conv-sentry-to-ga.tsx (375 lines)
- conv-sentry-to-ga.test.js (12 lines)
- fixtures.ts (211 lines)

The data flow is now direct: error-capture.tsx formats errors → ga.tsx tracks to GA. No intermediate conversions.

Commit: 8c72e63
</comment_old>

<comment_old>

<author>@yurishkuro</author>
@copilot please go critically through ga.tsx and error-capture.tsx and make sure only needed functions are included in error-capture.tsx. For example, ga.tsx imports breadcrumbsIntegration but never uses it.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot please go critically through ga.tsx and error-capture.tsx and make sure only needed functions are included in error-capture.tsx. For example, ga.tsx imports breadcrumbsIntegration but never u...

Reviewed and removed unnecessary code:

1. **Removed `breadcrumbsIntegration()`** - Was just a pass-through wrapper that returned config unchanged
2. **Simplified `init()`** - Now accepts breadcrumb config directly via `breadcrumbs` option instead of `integrations` array

All exports from error-capture.tsx are now actually used by ga.tsx:
- `IGAErrorData` - type for error data (line 116)
- `BrowserClient` - for context field (line 45)
- `captureException` - captures errors (line 175)
- `init` - initializes error capture (line 158)
- `trackNavigation` - tracks navigation (line 185)

Removed 9 lines of unnecessary wrapper code.

Commit: f731704
</comment_old>

<comment_new>
<comment_id>3672776017</comment_id>
<author>@yurishkuro</author>
@copilot in formatErrorForGA function, we first parsee stacktrace then format it again. Is that really necessary? Does GA accept some kind fo structured representation of a stack frame or just a string? If a string, why not just render the exception as it would be rendered if logged to console and submit it like that?

  const frames = parseStackTrace(error);

```
  const stack = formatStack(frames);
</comment_new>
```

</pr_comments>

</comments>

The last **5** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Summary

I've successfully addressed the feedback to simplify the stack trace

handling in the error capture implementation.

What Was Done

Problem identified: The code was unnecessarily parsing error stack traces into structured `IStackFrame[]` objects, then formatting them back into strings. Since Google Analytics only accepts strings (not structured data), this was overcomplicated.

Changes made:

1. **Removed `parseStackTrace()` function** (37 lines) - Was parsing stack trace strings into structured objects with filename, function, lineno, colno fields

2. **Removed `IStackFrame` interface** - No longer needed without structured parsing

3. **Simplified `formatStack()` function** - Now works directly with the stack string:
   - **Before:** Parse string → structured IStackFrame[] objects → format back to string
   - **After:** Clean string directly with simple replacements (remove origin, remove /static/js/ prefix, collapse whitespace)

4. **Updated `formatErrorForGA()`** - Now calls `formatStack(error)` directly instead of `formatStack(parseStackTrace(error))`

Benefits

- **Removed 39 lines of code** (from 521 to 482 lines in error-capture.tsx)
- **Simpler logic** - Direct string manipulation instead of parse-then-format
- **No intermediate data structures** - Eliminates IStackFrame objects
- **Same functionality** - Still produces the same cleaned stack traces for GA
- **All tests pass** - 23/23 tracking tests successful
- **Build successful** - Production build completes without errors

The implementation is now more straightforward and efficient - we work directly with strings throughout, avoiding unnecessary parsing into structured objects when we just need string output anyway.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'jaegertracing/jaeger-ui' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/jaeger-ui/jaeger-ui (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-12-19T00:15:11.696Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Replace @sentry/browser with internal error capture implementation
</pr_title>

<pr_description>
Summary

Successfully replaced the `@sentry/browser` dependency with a minimal internal error capture implementation that directly produces Google Analytics formatted data.

Changes Made

- [x] Analyze @sentry/browser usage and eliminate all unnecessary code
- [x] Remove conv-sentry-to-ga.tsx conversion layer entirely
- [x] Integrate all error formatting logic directly into error-capture.tsx
- [x] Remove unused `breadcrumbsIntegration` wrapper function
- [x] Simplify stack trace handling - removed parse-then-format

````
approach
- [x] Remove @sentry/browser dependency from package.json
- [x] All tests pass (23/23 tracking tests)
- [x] Build successful

### Architecture Simplification

**Before (3 conversion steps):**
```
error-capture.tsx → conv-sentry-to-ga.tsx → ga.tsx
```

**After (direct data flow):**
```
error-capture.tsx → ga.tsx
```

### Key Changes

**Removed files:**
- `conv-sentry-to-ga.tsx` (375 lines)
- `conv-sentry-to-ga.test.js` (12 lines)
- `fixtures.ts` (211 lines)

**Removed unnecessary code:**
- `breadcrumbsIntegration()` wrapper function
- `parseStackTrace()` function - was parsing stack into
structured frames
- `IStackFrame` interface - no longer needed
- Complex frame-based stack formatting - replaced with simple
string manipulation

**Simplified stack trace handling:**
- **Before:** Parse stack string → structured IStackFrame[] →
format back to string
- **After:** Clean/format stack string directly (no intermediate
parsing)

**Final implementation:**
- **error-capture.tsx** (482 lines, reduced from 521) - Directly
produces GA-formatted data
- **ga.tsx** (198 lines) - Receives pre-formatted GA data via
`onError` callback

### Exports from error-capture.tsx (all necessary):
1. **`IGAErrorData`** - Type for GA-formatted error data
2. **`BrowserClient`** - Mock class for type compatibility
````

```
3. **`captureException`** - Capture and format errors
4. **`init`** - Initialize error capture with callback
5. **`trackNavigation`** - Track navigation breadcrumbs

### Benefits

- Eliminated 598 lines across 3 removed files
- Removed 48 lines of unnecessary parsing/formatting code
- Simplified stack trace handling (string-to-string instead of
parse-format-stringify)
- No intermediate data structures
- All error formatting logic in one place
- Simpler, more maintainable code
- ~200KB bundle reduction

### Testing

✅ All 23 tracking tests pass
✅ TypeScript compilation successful
✅ Production build successful




</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@yurishkuro</author>
@copilot I see you did 1 to 1 equivalence, but the functionality
of IException and Breadcrumb is only used in conv-sentry-to-
ga.tsx, so this just introduces extra indirection. Can we not
simply call error-capture.tsx from tracking/ga.tsx directly and
only implement minimal functionality that is actually required
by ga.tsx?
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot I see you did 1 to 1 equivalence, but the
functionality of IException and Breadcrumb is only used in conv-
```

sentry-to-ga.tsx, so this just introduces extra indirection. Can we not simply call e...

You're right - I've simplified the implementation:

1. Moved type definitions (IException, IBreadcrumb, IStackFrame, IStacktrace) to conv-sentry-to-ga.tsx where they're actually used
2. Reduced error-capture.tsx from 381 to 300 lines, keeping only the minimal functionality needed by ga.tsx:
   - Error capture and breadcrumb tracking
   - Integration setup
   - Navigation tracking
3. Removed the unnecessary type abstraction layer

The types are now defined directly in conv-sentry-to-ga.tsx, and error-capture.tsx provides only the runtime functionality that ga.tsx requires. This eliminates the extra indirection while maintaining all functionality.

Commit: 74dd64e
</comment_old>

<comment_old>
<author>@yurishkuro</author>
@copilot This is still too much code. You have packages/jaeger-ui/src/utils/tracking/error-capture.tsx defining some types, then packages/jaeger-ui/src/utils/tracking/conv-sentry-to-ga.tsx converting them to some other types, then packages/jaeger-ui/src/utils/tracking/ga.tsx doing yet another conversion to package the data. Can we not remove conv-sentry-to-ga.tsx completely, and in error-capture.tsx define only the pieces of data that ga.tsx actually needs?
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot This is still too much code. You have packages/jaeger-ui/src/utils/tracking/error-capture.tsx defining some types, then packages/jaeger-ui/src/utils/tracking/conv-sentry-to-ga.tsx converting ...

Completely removed conv-sentry-to-ga.tsx and integrated all conversion logic directly into error-capture.tsx. The error capture now:

1. Directly produces GA-formatted data (`IGAErrorData` with message, category, action, label, value)
2. Calls the `onError` callback with ready-to-use GA data
3. Eliminates the conversion layer entirely

Removed files:
- conv-sentry-to-ga.tsx (375 lines)
- conv-sentry-to-ga.test.js (12 lines)
- fixtures.ts (211 lines)

The data flow is now direct: error-capture.tsx formats errors → ga.tsx tracks to GA. No intermediate conversions.

Commit: 8c72e63
</comment_old>

<comment_old>
<author>@yurishkuro</author>
@copilot please go critically through ga.tsx and error-capture.tsx and make sure only needed functions are included in error-capture.tsx. For example, ga.tsx imports breadcrumbsIntegration but never uses it.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot please go critically through ga.tsx and error-capture.tsx and make sure only needed functions are included in error-capture.tsx. For example, ga.tsx imports breadcrumbsIntegration but never u...

Reviewed and removed unnecessary code:

1. **Removed `breadcrumbsIntegration()`** - Was just a pass-through wrapper that returned config unchanged
2. **Simplified `init()`** - Now accepts breadcrumb config directly via `breadcrumbs` option instead of `integrations` array

All exports from error-capture.tsx are now actually used by ga.tsx:
- `IGAErrorData` - type for error data (line 116)
- `BrowserClient` - for context field (line 45)
- `captureException` - captures errors (line 175)
- `init` - initializes error capture (line 158)
- `trackNavigation` - tracks navigation (line 185)

Removed 9 lines of unnecessary wrapper code.

Commit: f731704
</comment_old>

<comment_old>
<author>@yurishkuro</author>
@copilot in formatErrorForGA function, we first parsee stacktrace then format it again. Is that really necessary? Does GA accept some kind fo structured representation of a stack frame or just a string? If a string, why not just render the exception as it would be rendered if logged to console and submit it like that?

  const frames = parseStackTrace(error);
  const stack = formatStack(frames);
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot in formatErrorForGA function, we first parsee stacktrace then format it again. Is that really necessary? Does GA accept some kind fo structured representation of a stack frame or just a strin...

You're right - the parse-then-format approach was unnecessary. GA only accepts strings, not structured data.

Simplified stack trace handling:
- **Before:** Parse stack string → IStackFrame[] → format back to string
- **After:** Clean/format stack string directly with simple string operations

Removed:
- `parseStackTrace()` function (37 lines)
- `IStackFrame` interface
- Complex frame-based formatting logic

The new `formatStack()` just cleans the original stack string by removing origin URLs and /static/js/ prefixes, then collapses whitespace - much simpler.

Reduced error-capture.tsx from 521 to 482 lines (39 lines saved).

Commit: 0e62d6a
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>packages/jaeger-ui/src/utils/tracking/error-capture.tsx:17</file>
<original_commit>0e62d6a</original_commit>

<comment_new>
<comment_id>2633090328</comment_id>
<author>@yurishkuro</author>
there is no compatibility requirements, delete this
</comment_new>

</comment_thread>

<comment_thread>
<file>packages/jaeger-ui/src/utils/tracking/error-capture.tsx:24</file>
<original_commit>0e62d6a</original_commit>

<comment_new>
<comment_id>2633095971</comment_id>
<author>@yurishkuro</author>
there are no variations or customizations we require, remove this and just hardcode the settings and/or behavior
</comment_new>

</comment_thread>

<comment_thread>
<file>packages/jaeger-ui/src/utils/tracking/error-capture.tsx:7</file>
<original_commit>0e62d6a</original_commit>

<comment_new>
<comment_id>2633098576</comment_id>
<author>@yurishkuro</author>
this module should not be GA aware, in theory we could use it with non-GA implementations.
</comment_new>

</comment_thread>

<comment_thread>
<file>packages/jaeger-ui/src/utils/tracking/error-capture.tsx:275</file>
<original_commit>0e62d6a</original_commit>

<comment_new>
<comment_id>2633103666</comment_id>
<author>@yurishkuro</author>
this function should be in ga.tsx module if it's specific to GA
</comment_new>

</comment_thread>

<comment_thread>
<file>packages/jaeger-ui/src/utils/tracking/error-capture.tsx:477</file>
<original_commit>0e62d6a</original_commit>

<comment_new>
<comment_id>2633106817</comment_id>
<author>@yurishkuro</author>
There are no compatiblity requirements. REmove if not needed.
</comment_new>

</comment_thread>

<comment_thread>
<file>packages/jaeger-ui/src/utils/tracking/error-capture.tsx:4</file>
<original_commit>0e62d6a</original_commit>

<comment_new>
<comment_id>2633109884</comment_id>
<author>@yurishkuro</author>
"minimal" is quite a stretch. Write a header comment documenting what each function does and show examples of how it helps in reporting data for tracking.
</comment_new>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>3672829709</comment_id>

```
<author>@yurishkuro</author>
@copilot address all review comments in the PR
</comment_new>
```

```
</pr_comments>

</comments>

----
The last **6** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>
```

```
Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>
```

```
A **CI/Build Failure** has been mentioned in the problem statement,
make sure to follow your instructions for addressing CI/Build
Failures.
```